

National Park Insights: An ETL Pipeline for Alerts, Activities, and Trip Planning Using Prefect

DSAN 5500: Data Structures, Objects, and Algorithms in Python

Gentry Lamb

Abstract

This project presents the design and implementation of an ETL (Extract, Transform, Load) pipeline built to automate collecting, processing, and delivering timely insights from the National Park Service (NPS) API. Motivated by personal experiences planning trips across 21 national parks, the project addresses a persistent challenge for outdoor enthusiasts: fragmented, inconsistent access to critical park information such as closures, alerts, and available activities. While the NPS API offers structured access to this data, it remains difficult to use effectively without a more accessible interface or automation.

The developed solution leverages the Python-based orchestration tool Prefect to manage tasks and flows, ensuring that data pipelines run reliably and can be scheduled for recurring updates. Raw data from the NPS API is extracted across three key domains (parks, alerts, and activities) and then validated using Pydantic to ensure consistency and data quality. Alerts are further enriched through sentiment analysis and a custom severity scoring system, enabling travelers to filter and prioritize alerts based on urgency. The resulting structured datasets are saved in `.jsonl` format for easy reuse, analysis, and visualization.

A key deliverable is a weekly Markdown report summarizing the most relevant alerts and changes, along with an interactive map to visualize national park status. While this implementation is focused on a Minimum Viable Product (MVP), it provides a scalable foundation for future enhancements such as real-time alerting, itinerary planning, or personalized recommendations. This project demonstrates how automated data pipelines can bridge the gap between raw API data and actionable insights for end users, using modern tools in Python to improve decision-making and trip planning for outdoor adventurers and public land users.

Motivation

Known for its natural beauty and vast wilderness, Montana is home to two of the most iconic national parks in the United States. Growing up in this awe-inspiring state instilled a deep appreciation for nature and a lifelong commitment to spending time outdoors. After moving to Colorado for college, that commitment evolved into a personal goal: to complete a backpacking trip in every U.S. national park. I have completed 21 thus far, exactly one-third of the current federally designated sites. Planning those trips has become as much a part of the experience as the trails themselves. Balancing routes, permits, weather forecasts, and increasingly, real-time alerts about fire danger, road closures, and other unexpected events is a large part of making these trips a reality.

That planning process is often frustrating at times. Information is scattered across park-specific websites, not always up to date, and hard to monitor consistently. I found myself checking the National Park Service (NPS) websites over and over before a trip, just to make sure I hadn't missed a sudden trail closure or hazardous condition. That's when I discovered the NPS API (2017), which makes structured data on park alerts, activities, and general park info accessible through a RESTful interface. It felt like the missing link, raw access to the exact data I needed, but with none of the user-friendly polish to make it practical for regular use.

This project grew out of a desire to fill that gap. I wanted to build something that didn't just pull park data for its own sake, but that could help travelers like me make informed decisions. With that goal, I designed an ETL (Extract, Transform, Load) pipeline in Python that could automatically turn information on park websites into usable insights for travelers and planners. This project not only fulfills a personal goal - to create a better tool for planning future trips to national parks - but also serves as a testbed for applying best practices in pipeline design, including orchestration, scheduling, data validation, and artifact generation. The system is built to be extensible, too. Future iterations could include personalized park filters, travel-ready itineraries, or real-time notifications. For now, the core functionality offers something simple but impactful: regular and reliable insight into the changing conditions across our national parks.

Problem Statement

As mentioned above, the current method for gathering information on national parks, such as alerts and activity availability, is largely manual. Users must visit multiple NPS web pages individually to get up-to-date information. While the National Park Service (NPS) API offers structured access to data on alerts, activities, and park information, the data can be difficult to understand in its raw form. It lacks a user-friendly interface and doesn't solve the core problem of timely, consolidated delivery. That's where an ETL pipeline can help. By automating the collection, transformation, and distribution of this information, the pipeline reduces manual effort and creates a more reliable, centralized source of truth.

There were a myriad of ideas I wanted to implement in this project, but given the time constraints of the semester, I had to narrow in on a Minimum Viable Product (MVP). The MVP focuses on using [Prefect](#), a Python-based workflow orchestration tool, to create a pipeline that extracts data from the NPS API, validates it with [Pydantic](#), transforms it into traveler-relevant summaries, and loads it to a database so that it can be used later. The pipeline is scheduled to execute weekly, producing two main deliverables: a Markdown report summarizing the various suggestions and an interactive map showing the most recent park alerts. This MVP addresses the core workflow inefficiencies while providing a strong foundation for future expansion.

Extract, Transform, Load (ETL) Pipeline

The core of this project implements a robust and automated ETL pipeline that collects, cleans, enriches, and stores data from the National Park Service API. The goal of this pipeline is to take raw, unprocessed data about sites owned by the NPS and turn it into meaningful, organized, and ready-to-use information for further analysis or reporting. This process is automated using [Prefect](#), which helps break the workflow into smaller, manageable tasks that can be monitored, run on a schedule, and easily extended in the future.

Extract - Gathering the Data

A key strength of this pipeline lies in its ability to consistently extract, clean, and validate three interconnected datasets from the NPS API. These datasets include parks, alerts, and activities. Each of these serves a distinct purpose but are designed to integrate smoothly into a unified data pipeline.

The parks dataset provides foundational information about each site managed by the NPS. This includes the park's name, park code, location (including state and geographic coordinates), a short descriptive overview, its designation (such as "National Park," "Historic Site," or "National Seashore."), and various other features. This dataset anchors the pipeline, supplying the essential context for any mapping, alert monitoring, or activity planning feature built downstream.

The alerts dataset was the primary focus initially as I wanted to gather data that changed significantly day-to-day. It includes time-sensitive messages issued by the NPS to communicate urgent or important updates about park conditions. These messages range from weather closures and fire warnings to temporary restrictions, safety tips, and general updates on services or amenities. The content is often unstructured and highly variable in tone and detail. Because of this variability, additional downstream processing is applied later in the pipeline to assess each alert's sentiment and severity.

Finally, the activities dataset catalogs the range of recreational and educational opportunities available across the park system. Activities can include hiking, camping, boating, wildlife

viewing, ranger-led tours, and more. Each entry links one or more activities to specific parks, forming a crucial bridge between park information and user preferences.

Once these datasets are extracted from the API, they undergo a structured data validation process. Rather than loading the raw data directly into spreadsheets or data frames, this pipeline uses a more robust and maintainable approach by defining formal data models using Pydantic, a Python library that enforces data types and rules. For each dataset, a dedicated class is defined. These classes lay out the expected fields, their required formats (such as strings, floats, or lists), and any default values to use when information is missing or malformed. This validation layer serves as a powerful early-warning system. For example, if a park's location data is missing or incorrectly formatted, the model catches it immediately and either supplies a default value or safely omits the record, depending on the use case. Similarly, if an alert is missing a description or an activity is assigned an invalid name, these issues are surfaced and handled during validation, not later when they might cause errors in analysis.

Transform - Enriching the Data

While all three datasets are cleaned and formatted, the alerts dataset is given special attention because it's the most dynamic and unstructured. These alerts often contain free-text descriptions that vary widely in tone, importance, and length. To make them more useful, the pipeline performs two key transformations.

Sentiment Analysis

The pipeline evaluates the emotional tone of each alert using a tool that measures sentiment. This helps identify whether an alert is neutral, negative (e.g., a warning or danger), or slightly positive by assigning a value between negative 1 and positive 1. For example, an alert that says "Caution: Recent bear sightings" would register as more negative than something like "Information: Parking lot construction." This helps quantify how urgent or concerning an alert might feel to a visitor.

Severity Scoring

Each alert also comes with a predefined category assigned by the NPS, such as "Information," "Caution," "Park Closure," or "Danger." These categories are mapped to severity scores based on how serious they are. For instance, a "Park Closure" or "Danger" alert is scored higher than a general "Information" alert. While there is no definitive answer for how severe an alert is, I use values in the range of 1-5 as a basis.

Final Score

To make alerts easier to sort and prioritize, the pipeline combines the sentiment and severity into a single composite score. This final score provides a way to rank alerts in terms of overall urgency and importance, taking into account both what the alert says and how it's likely to affect visitors. This enrichment process transforms plain text into something that can be compared, filtered, and visualized. This makes it far more useful than the raw descriptions alone.

Load - Saving the Outputs

Once all data is cleaned, validated, and transformed, it is saved into structured files (in `.jsonl` format). These files serve as the project's internal "database" and are well-organized, easy to read, and compatible with many tools for analysis or reporting. Each of the three datasets (parks, alerts, and activities) are saved separately. This modular approach allows different parts of the data to be reused or analyzed independently depending on the needs of the application. Early in the project, there was consideration of using a more traditional database system, such as MongoDB for flexible document storage or SQL for structured relational queries. However, given the relatively modest size of the datasets and the timeline constraints of the project, it became clear that implementing a full database was not necessary at this stage. Instead, the decision to use `.jsonl` files prioritized transparency and simplicity. This approach enabled faster iteration and easier debugging, while still leaving open the possibility of scaling up to a database-backed architecture later.

Automation with Prefect

To ensure that the entire pipeline, from data extraction to transformation, validation, and final loading, runs in a reliable and repeatable way, I used [Prefect](#), as it provides a clear and scalable way to define workflows using two key building blocks: **tasks** and **flows**. Each key operation in the ETL process is defined as a **task** in Prefect. For example:

- The `get_parks()` task queries API to gather information about parks.
- The `transform_alerts()` task takes validated alerts, adding information on alert sentiment, severity, and score.
- The `load_into_db()` task handles serialization and saving of the final cleaned data to disk.

These tasks are then composed into **flows**, higher-level processes that determine the order and logic of task execution. For instance, the main `scrape_NPS()` flow coordinates the full process of extracting, validating, and saving data for all three datasets: parks, alerts, and activities. Prefect handles this orchestration automatically, so that tasks are executed in the proper sequence and only rerun when necessary.

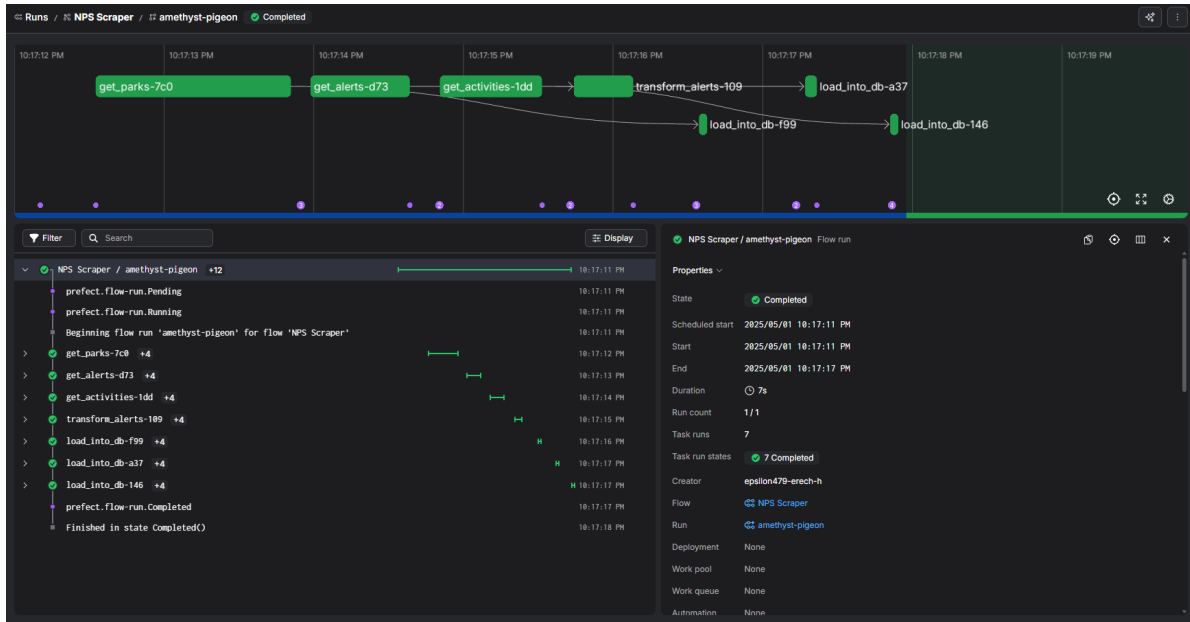


Figure 1: Example Run Output of NPS Scraper in Prefect

Using Prefect provided several key benefits that helped shape the success and maintainability of the pipeline. It enabled easy automation, allowing flows to be scheduled for regular execution, such as weekly updates, without the need for manual triggers. Further, Prefect’s built-in user interface offered robust monitoring and logging capabilities. Each task’s status could be tracked in real-time, and logs were automatically generated, making it easy to catch and debug issues like failed downloads or other discrepancies. With each task defined independently, new functionality could be introduced with minimal disruption to the existing flow. Beyond these benefits, Prefect also made development more flexible and less error-prone. I was able to experiment with and test tasks in isolation before integrating them into the full pipeline, which made debugging more straightforward and helped ensure that each component worked correctly on its own. This modular and organized approach significantly enhanced the clarity and reliability of the overall project.

Artifacts and Useful Output

Beyond processing and saving structured data, the pipeline also produces two final artifacts. These user-facing outputs represent the culmination of the ETL process and its real-world utility. To generate these artifacts, a second Prefect flow was created specifically for the reporting and visualization steps. Each artifact highlights a different aspect of the data and serves a distinct purpose for end users. An example run of this flow from the Prefect UI is shown below.

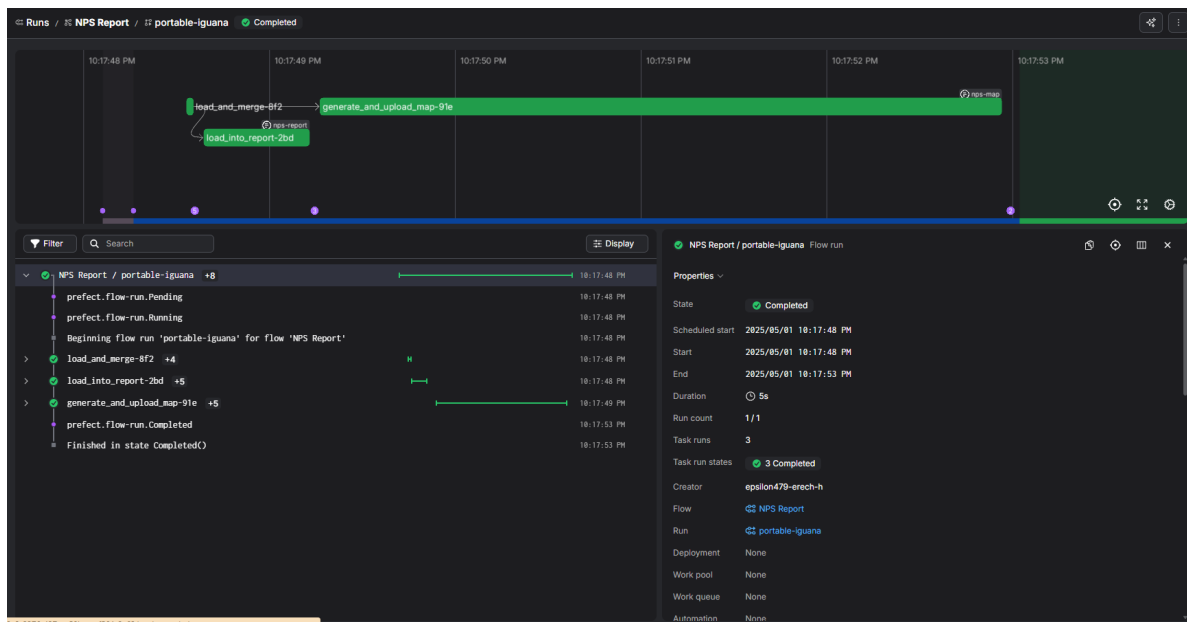


Figure 2: Example Run Output of NPS Report in Prefect

Markdown Report

The first artifact is a generated Markdown report, built by pulling insights from the processed park and alert data. This report includes several key summaries and visual highlights: the total number of alerts across all parks, a breakdown of the most common alert types, and a ranked list of parks experiencing the highest number of active alerts. It also includes detailed tables showing parks ranked on different metrics, with direct links to the park pages. Behind the scenes, generating this report required substantial data transformation like grouping, filtering, and summarizing. Functions like `aggregate_data()` and `get_top_sites()` were developed to handle these transformations cleanly and consistently. The final report is written to markdown using a custom function and is designed to be portable. It can be embedded directly into a website, emailed as part of a weekly digest, or converted into HTML or PDF for broader distribution. Since it is regenerated automatically whenever the pipeline runs, it provides accurate updates about conditions across the National Park system, without requiring manual reporting or analysis. Below is an example of this report.

NPS Scraping Report

Report Date: 02-May-2025

Total Alerts Collected: 645

Site Type: National Park or Preserve

Most Common Alert Types

The most frequently occurring alert types were:

Category	Count
Information	327
Park Closure	182
Caution	122
Danger	14

National Park or Preserve with the Most Alerts

	Site Name	State(s)	Site Website	Avg. Sentiment	Avg. Severity	Number of Alerts
1	Everglades National Park	FL	https://www.nps.gov/everglades.htm	0.211978	1.72222	9
2	Shenandoah National Park	VA	https://www.nps.gov/shenandoah.htm	0.0252875	0.75	8
3	Cuyahoga Valley National Park	OH	https://www.nps.gov/cuyahogavalley.htm	0.00995714	2	7
4	Carlsbad Caverns National Park	NM	https://www.nps.gov/carlsbadcaverns.htm	0.164714	0.571429	7
5	North Cascades National Park	WA	https://www.nps.gov/northcascades.htm	0.0229	2.07143	7
6	Saguaro National Park	AZ	https://www.nps.gov/saguaro.htm	0.0896	0.8	5
7	Zion National Park	UT	https://www.nps.gov/zion.htm	0.19254	1.6	5
8	White Sands National Park	NM	https://www.nps.gov/whitesands.htm	0.16462	1	5
9	Lassen Volcanic National Park	CA	https://www.nps.gov/lassen.htm	0.19908	1.7	5
10	Mount Rainier National Park	WA	https://www.nps.gov/mount-rainier.htm	0.02632	2.1	5

National Park or Preserve with Highest Alert Scores

Using a scoring system that adjusts for both alert type and sentiment, we've identified the 10 sites currently facing the most serious issues:

	Site Name	State(s)	Site Website	Avg. Sentiment	Avg. Severity	Alert Score
1	North Cascades National Park	WA	https://www.nps.gov/noca/00222x.htm	0.0095714	2.07143	100
2	Cuyahoga Valley National Park	OH	https://www.nps.gov/cova/0095714.htm	0.0095714	2	96.17
3	Lassen Volcanic National Park	CA	https://www.nps.gov/lavo/019908x.htm	0.19908	1.7	79.2
4	Sequoia & Kings Canyon National Parks	CA	https://www.nps.gov/seki/0305133x.htm	0.305133	3	76.05
5	Guadalupe Mountains National Park	TX	https://www.nps.gov/guam/0532367x.htm	0.532367	2.16667	72.67
6	Everglades National Park	FL	https://www.nps.gov/ever/02211078x.htm	0.2211078	1.72222	70.5
7	Hawai i Volcanoes National Park	HI	https://www.nps.gov/havo/0071105x.htm	0.0071105	3	67.71
8	Zion National Park	UT	https://www.nps.gov/zion/0119254x.htm	0.119254	1.6	66.62
9	Mount Rainier National Park	WA	https://www.nps.gov/mora/002632x.htm	0.02632	2.1	66.48
10	Mojave National Preserve	CA	https://www.nps.gov/moja/00550833x.htm	0.0550833	3	60.83

National Park or Preserve with Highest Alert Scores

Conversely, these 10 sites are currently facing the fewest and least serious issues:

	Site Name	State(s)	Site Website	Avg. Sentiment	Avg. Severity	Alert Score
1	Gates Of The Arctic National Park & Preserve	AK	https://www.nps.gov/gaar/004767x.htm	0.4767	0.5	0
2	Isle Royale National Park	MI	https://www.nps.gov/isro/006597x.htm	0.6597	1	0.52

	Site Name	State(s)	Site Website	Avg. Sentiment	Avg. Severity	Alert Score
3	Joshua Tree National Park	CA	https://www.nps.gov/jotr/index.htm	0.6249	1	0.75
4	Death Valley National Park	CA,NV	https://www.nps.gov/deva/index.htm	0.125	0.5	0.75
5	Great Sand Dunes National Park & Preserve	CO	https://www.nps.gov/grsa/index.htm	0.6167	1	0.8
6	Valles Caldera National Preserve	NM	https://www.nps.gov/valn/index.htm	0.2263	0.5	0.83
7	Grand Teton National Park	WY	https://www.nps.gov/grte/index.htm	0.2263	0.5	0.83
8	Hot Springs National Park	AR	https://www.nps.gov/hosp/index.htm	0.2263	0.5	1.58
9	New River Gorge National Park & Preserve	WV	https://www.nps.gov/neri/index.htm	0.2263	0.5	1.58
10	Haleakalā National Park	HI	https://www.nps.gov/hale/index.htm	0.2263	0.5	1.58

Popular Activities at Top National Park or Preserve

Great Sand Dunes National Park & Preserve

- Arts and Culture
- Astronomy
- Auto and ATV
- Biking
- Camping
- Climbing
- Compass and GPS
- Fishing
- Food
- Guided Tours
- Hands-On
- Hiking
- Horse Trekking
- Hunting and Gathering
- Junior Ranger Program
- Museum Exhibits

- Park Film
- Shopping
- Skiing
- Snow Play
- Snowshoeing
- Swimming
- Tubing
- Water Skiing
- Wildlife Watching

Gates Of The Arctic National Park & Preserve

- Camping
- Climbing
- Hiking
- Hunting and Gathering
- Junior Ranger Program
- Paddling

Death Valley National Park

- Astronomy
- Biking
- Camping
- Canyoneering
- Flying
- Food
- Golf
- Guided Tours
- Hiking
- Horse Trekking
- Junior Ranger Program
- Living History
- Museum Exhibits
- Park Film
- Shopping
- Wildlife Watching

Isle Royale National Park

- Arts and Culture

- Astronomy
- Boating
- Camping
- Fishing
- Flying
- Food
- Guided Tours
- Hiking
- Junior Ranger Program
- Museum Exhibits
- Paddling
- Park Film
- SCUBA Diving
- Shopping
- Wildlife Watching

Joshua Tree National Park

- Astronomy
- Auto and ATV
- Biking
- Camping
- Climbing
- Food
- Guided Tours
- Hiking
- Horse Trekking
- Junior Ranger Program
- Museum Exhibits
- Shopping
- Wildlife Watching

Interactive Alert Map

The second, and more technically complex, artifact is an interactive map showing all parks that currently have active alerts. This map was created using Python plotting libraries and geographic coordinates from the alerts dataset. The steps involved were:

1. Generating an interactive map with alerts plotted using `folium`
2. Saving the plot locally as an HTML
3. Accessing my domain and hosting the HTML there
4. Retrieving and storing the URL of the hosted plot to make it accessible to users

This process of generating a link artifact, a dynamic reference to an externally hosted image, was not covered in most Prefect tutorials and required some custom logic to attach the image's URL as metadata to the flow run. The artifact value is the visual and geographic dimension it adds to the data. Users can quickly understand which regions are affected by alerts without scanning a long list. The artifact is linked here: [Interactive Alert Map](#)

Future Work

This project was scoped to deliver a functional MVP pipeline that automates the extraction and processing of key data from the NPS API. However, several enhancements could significantly expand its utility:

- **User-Centric Alerts and Notifications:** Integrating a notification system (via email or SMS) would allow users to subscribe to parks or states of interest and receive real-time alerts when critical events (e.g., wildfires, road closures) are issued.
- **Dynamic Itinerary Planning:** The next step is to build a front-end application or dashboard that lets users input travel dates, preferred activities, and locations, then receive personalized recommendations and route plans using the clean datasets from this pipeline.
- **Web-Based Dashboard:** The Markdown report is a practical output, but hosting a real-time dashboard using tools like Streamlit or Dash would provide a more interactive and scalable user interface, especially for non-technical users.
- **Database Integration:** Storing processed data in a database system like SQL or MongoDB would facilitate more robust querying, historical data tracking, and multi-user access for collaborative travel planning or analytics.
- **Expanded Enrichment Pipeline:** Incorporating third-party data, such as weather forecasts, wildfire zones, or trail conditions, could help contextualize NPS alerts and increase planning accuracy.

Conclusion

The NPS ETL pipeline bridges a practical gap between structured public data and real-world decision-making. By automating the collection and transformation of park alerts, activities, and metadata, the project makes it easier for travelers to stay informed, plan safer trips, and engage more deeply with public lands. Leveraging Prefect, Pydantic, and modern Python tools, the pipeline ensures data integrity, repeatability, and transparency.

Though this version focuses on alert summaries and basic park activity classification, it lays the groundwork for scalable enhancements. This project not only demonstrates how public data can be transformed into actionable insights but also how workflows can greatly enhance this process.

References

2017. *National Parks Service*. U.S. Department of the Interior. <https://www.nps.gov/subjects/developer/api-documentation.htm>.

Appendix

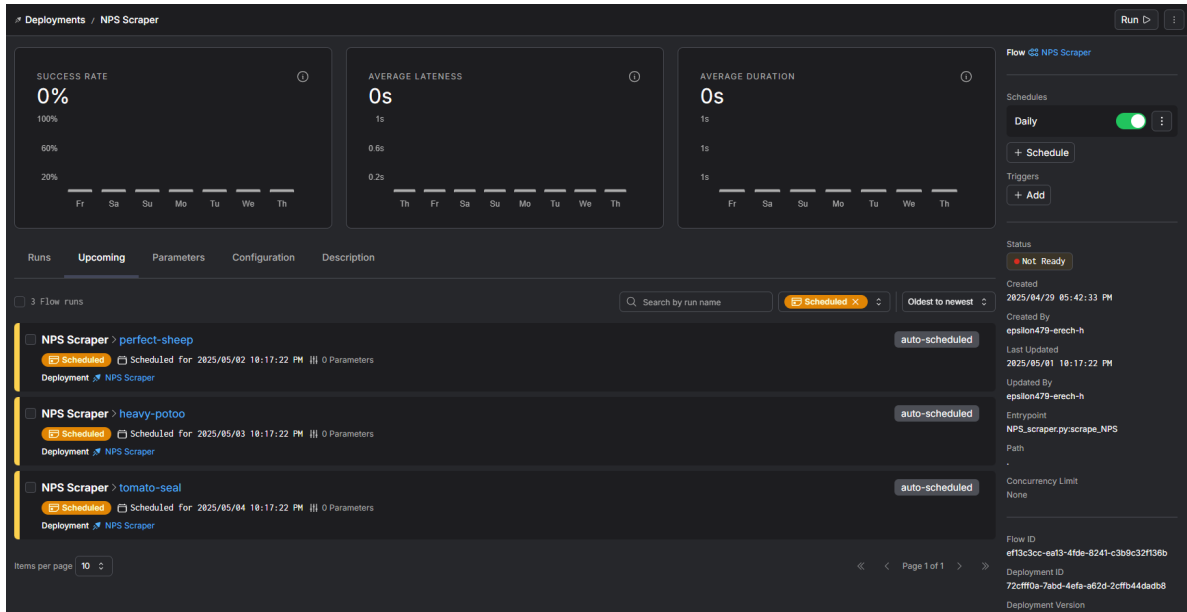


Figure 3: NPS Scraper Deployment in Prefect

Complete Code

```
# Libraries to extract api keys
from google.colab import userdata
```

```
# Get API keys
prefect_api_key = userdata.get("PREFECT_API_KEY")
nps_api_key = userdata.get("NPS_API_KEY")
```

```
%%capture
!pip install prefect
```

```
!prefect cloud login -k "$prefect_api_key"
```

```
# Libraries used in extraction
from pydantic import BaseModel, HttpUrl, field_validator, FilePath
from nltk.sentiment.vader import SentimentIntensityAnalyzer
from prefect.task_runners import ThreadPoolTaskRunner
```

```

from prefect.futures import wait
from prefect import flow, task
from typing import List
import pandas as pd
import requests
import string
import json
import nltk

# Download the VADER lexicon for sentiment analysis (quietly)
nltk.download('vader_lexicon', quiet=True)

# Define Pydantic model for parks
class Park(BaseModel):
    fullName: str | None
    parkCode: str | None
    states: str | List[str] | None
    latitude: float | None
    longitude: float | None
    designation: str | None
    url: HttpUrl | None

    # Field validator to handle blank values for all fields
    @field_validator('*', mode='before')
    @classmethod
    def clean_blanks(cls, value: str | float):
        if value == '':
            return None
        return value

# Define Pydantic model for activities
class Activity(BaseModel):
    parkCode: str | None
    activity: List[str] | None

    # Field validator to handle blank values for activity fields
    @field_validator('*', mode='before')
    @classmethod
    def clean_blanks(cls, value: str | float):
        if value == '':
            return None
        return value

```



```

# Define Pydantic model for alerts
class Alert(BaseModel):
    title: str | None
    description: str | None
    parkCode: str | None
    category: str | None
    sentiment: float = None
    severity: float = None
    score: float = None

    # Field validator to handle blank values for alert fields
    @field_validator('*', mode='before')
    @classmethod
    def clean_blanks(cls, value: str | float):
        if value == '':
            return None
        return value

# Function to extract data from the NPS API
def extract_NPS(endpoint: HttpUrl, verbose: bool = False) -> List[dict]:
    """
    Extract data from the NPS API.

    Args:
        endpoint (HttpUrl): The URL of the API endpoint to retrieve data from.
        verbose (bool): A flag to print detailed information about the request.

    Returns:
        List[dict]: A list of dictionaries containing the API response data.
    """
    # Fetch data from the provided API endpoint
    global nps_api_key
    headers = {"X-API-Key": nps_api_key}
    response = requests.get(endpoint, headers=headers)
    data = response.json()

    # Print available data keys if verbose flag is set
    if verbose:
        print("Keys for request:\n", data.keys())
        print("Keys for 'data':\n", data["data"][0].keys())

    return data["data"]

```

```

# Function to get park data from the API and return as validated objects
@task
def get_parks(parks_endpoint: HttpUrl, verbose: bool = False) -> List[Park]:
    """
    Retrieve park data from the NPS API and return it as validated Park objects.

    Args:
        parks_endpoint (HttpUrl): The URL endpoint for fetching park data.
        verbose (bool): Flag to enable verbose logging of data retrieval.

    Returns:
        List[Park]: A list of validated Park Pydantic models.
    """
    parks_data = extract_NPS(parks_endpoint)
    validated_parks = [Park(**park) for park in parks_data]

    return validated_parks

# Function to get alert data from the API and return as validated objects
@task
def get_alerts(alerts_endpoint: HttpUrl, verbose: bool = False) -> List[Alert]:
    """
    Retrieve alert data from the NPS API and return it as validated Alert objects.

    Args:
        alerts_endpoint (HttpUrl): The URL endpoint for fetching alert data.
        verbose (bool): Flag to enable verbose logging of data retrieval.

    Returns:
        List[Alert]: A list of validated Alert Pydantic models.
    """
    alerts_data = extract_NPS(alerts_endpoint)
    validated_alerts = [Alert(**alert) for alert in alerts_data]

    return validated_alerts

# Function to get activity data from the API, process it, and return as validated objects
@task
def get_activities(activities_endpoint: HttpUrl, verbose: bool = False) -> List[Activity]:
    """
    Retrieve activity data from the NPS API, process it into a long format,
    and return it as validated Activity objects.

```

```

Args:
    activities_endpoint (HttpUrl): The URL endpoint for fetching activity data.
    verbose (bool): Flag to enable verbose logging of data retrieval.

Returns:
    List[Activity]: A list of validated Activity Pydantic models.
"""
activities_data = extract_NPS(activities_endpoint)
# Unpack activities into a long format DataFrame
df = pd.DataFrame([
    {'activity': activity['name'], 'parkCode': park['parkCode']}
    for activity in activities_data
    for park in activity['parks']
])
# Group activities by parkCode
df_grouped = df.groupby("parkCode")["activity"].agg(list).reset_index()
validated_activities = [Activity(**row) for row in df_grouped.to_dict(orient="records")]

return validated_activities

# Function to transform alert data by calculating sentiment, severity, and score
@task
def transform_alerts(alert_list: List[Alert]) -> List[Alert]:
    """
    Transform alert data by calculating sentiment, severity, and score for each alert.

    Args:
        alert_list (List[Alert]): A list of Alert objects to be processed.

    Returns:
        List[Alert]: A list of transformed Alert objects with added sentiment, severity, and
    """
    category_weights = {
        'Caution': 1,
        'Information': 0.5,
        'Park Closure': 3,
        'Danger': 5
    }
    # Initialize sentiment analysis
    analyzer = SentimentIntensityAnalyzer()
    # Calculate sentiment, severity, and score for each alert
    for alert_obj in alert_list:

```

```

        sentiment = analyzer.polarity_scores(alert_obj.description)['compound']
        severity = category_weights.get(alert_obj.category, 0)
        score = round(severity * (1 - sentiment), 3)
        # Add computed attributes to the alert object
        alert_obj.sentiment = sentiment
        alert_obj.severity = severity
        alert_obj.score = score

    return alert_list

# Function to load data into a file (acting as a database here)
@task
def load_into_db(data: List[BaseModel], db_fpath: FilePath) -> None:
    """
    Load a list of data (as Pydantic models) into a file.

    Args:
        data (List[BaseModel]): A list of Pydantic model objects to be saved.
        db_fpath (FilePath): The file path where the data will be saved.

    Returns:
        None
    """
    with open(db_fpath, 'w') as outfile:
        for item in data:
            outfile.write(item.model_dump_json() + '\n')

# Main function to scrape NPS data from multiple endpoints, process it, and save to files
@flow(name="NPS Scraper") #, task_runner=ThreadPoolTaskRunner()
def scrape_NPS(
    parks_endpoint: str = "https://developer.nps.gov/api/v1/parks?limit=500",
    alerts_endpoint: str = "https://developer.nps.gov/api/v1/alerts?limit=1000",
    activities_endpoint: str = "https://developer.nps.gov/api/v1/activities/parks",
    parks_filename: str = "extracted_parks.jsonl",
    alerts_filename: str = "extracted_alerts.jsonl",
    activities_filename: str = "extracted_activities.jsonl"
) -> None:
    """
    Main flow to scrape NPS data, process it, and save it into files.

    Args:
        parks_endpoint (str): The endpoint for fetching park data.
    """

```

```

        alerts_endpoint (str): The endpoint for fetching alert data.
        activities_endpoint (str): The endpoint for fetching activity data.
        parks_filename (str): The filename where park data will be saved.
        alerts_filename (str): The filename where alert data will be saved.
        activities_filename (str): The filename where activity data will be saved.

Returns:
    None
"""
# Submit tasks concurrently
extracted_parks = get_parks(parks_endpoint)
extracted_alerts = get_alerts(alerts_endpoint)
extracted_activities = get_activities(activities_endpoint)

# Now transform alerts (depends on extracted_alerts being ready)
transformed_alerts = transform_alerts(extracted_alerts)

# Save the extracted and transformed data to separate files
load_into_db(extracted_parks, parks_filename)
load_into_db(transformed_alerts, alerts_filename)
load_into_db(extracted_activities, activities_filename)

# Run the NPS scraping function
scrape_NPS()

```

```

%%writefile NPS_scraper.py
# Libraries used in extraction
from pydantic import BaseModel, HttpUrl, field_validator, FilePath
from nltk.sentiment.vader import SentimentIntensityAnalyzer
from prefect.task_runners import ThreadPoolTaskRunner
from prefect.futures import wait
from prefect import flow, task
from typing import List
import pandas as pd
import requests
import string
import json
import nltk

# Download the VADER lexicon for sentiment analysis (quietly)
nltk.download('vader_lexicon', quiet=True)

```

```

# Define Pydantic model for parks
class Park(BaseModel):
    fullName: str | None
    parkCode: str | None
    states: str | List[str] | None
    latitude: float | None
    longitude: float | None
    designation: str | None
    url: HttpUrl | None

    # Field validator to handle blank values for all fields
    @field_validator('*', mode='before')
    @classmethod
    def clean_blanks(cls, value: str | float):
        if value == '':
            return None
        return value

# Define Pydantic model for activities
class Activity(BaseModel):
    parkCode: str | None
    activity: List[str] | None

    # Field validator to handle blank values for activity fields
    @field_validator('*', mode='before')
    @classmethod
    def clean_blanks(cls, value: str | float):
        if value == '':
            return None
        return value

# Define Pydantic model for alerts
class Alert(BaseModel):
    title: str | None
    description: str | None
    parkCode: str | None
    category: str | None
    sentiment: float = None
    severity: float = None
    score: float = None

    # Field validator to handle blank values for alert fields

```

```

@field_validator('*', mode='before')
@classmethod
def clean_blanks(cls, value: str | float):
    if value == '':
        return None
    return value

# Function to extract data from the NPS API
def extract_NPS(endpoint: HttpUrl, verbose=False) -> List[dict]:
    # Fetch data from the provided API endpoint
    global nps_api_key
    headers = {"X-Api-Key": nps_api_key}
    response = requests.get(endpoint, headers=headers)
    data = response.json()

    # Print available data keys if verbose flag is set
    if verbose:
        print("Keys for request:\n", data.keys())
        print("Keys for 'data':\n", data["data"][0].keys())

    return data["data"]

# Function to get park data from the API and return as validated objects
@task
def get_parks(parks_endpoint: HttpUrl, verbose=False) -> List[Park]:
    parks_data = extract_NPS(parks_endpoint)
    validated_parks = [Park(**park) for park in parks_data]

    return validated_parks

# Function to get alert data from the API and return as validated objects
@task
def get_alerts(alerts_endpoint: HttpUrl, verbose=False) -> List[Alert]:
    alerts_data = extract_NPS(alerts_endpoint)
    validated_alerts = [Alert(**alert) for alert in alerts_data]

    return validated_alerts

# Function to get activity data from the API, process it, and return as validated objects
@task
def get_activities(activities_endpoint: HttpUrl, verbose=False) -> List[Activity]:
    activities_data = extract_NPS(activities_endpoint)

```

```

# Unpack activities into a long format DataFrame
df = pd.DataFrame([
    {'activity': activity['name'], 'parkCode': park['parkCode']}
    for activity in activities_data
    for park in activity['parks']
])
# Group activities by parkCode
df_grouped = df.groupby("parkCode")["activity"].agg(list).reset_index()
validated_activities = [Activity(**row) for row in df_grouped.to_dict(orient="records")]

return validated_activities

# Function to transform alert data by calculating sentiment, severity, and score
@task
def transform_alerts(alert_list: List[Alert]) -> List[Alert]:
    category_weights = {
        'Caution': 1,
        'Information': 0.5,
        'Park Closure': 3,
        'Danger': 5
    }
    # Initialize sentiment analysis
    analyzer = SentimentIntensityAnalyzer()
    # Calculate sentiment, severity, and score for each alert
    for alert_obj in alert_list:
        sentiment = analyzer.polarity_scores(alert_obj.description)['compound']
        severity = category_weights.get(alert_obj.category, 0)
        score = round(severity * (1 - sentiment), 3)
        # Add computed attributes to the alert object
        alert_obj.sentiment = sentiment
        alert_obj.severity = severity
        alert_obj.score = score

    return alert_list

# Function to load data into a file (acting as a database here)
@task
def load_into_db(data: List[BaseModel], db_fpath: FilePath) -> None:
    with open(db_fpath, 'w') as outfile:
        for item in data:
            outfile.write(item.model_dump_json() + '\n')

```



```

# Main function to scrape NPS data from multiple endpoints, process it, and save to files
@flow(name="NPS Scraper") #, task_runner=ThreadPoolTaskRunner()
def scrape_NPS(
    parks_endpoint: str = "https://developer.nps.gov/api/v1/parks?limit=500",
    alerts_endpoint: str = "https://developer.nps.gov/api/v1/alerts?limit=1000",
    activities_endpoint: str = "https://developer.nps.gov/api/v1/activities/parks",
    parks_filename: str = "extracted_parks.jsonl",
    alerts_filename: str = "extracted_alerts.jsonl",
    activities_filename: str = "extracted_activities.jsonl"
) -> None:
    # Submit tasks concurrently
    extracted_parks = get_parks(parks_endpoint)
    extracted_alerts = get_alerts(alerts_endpoint)
    extracted_activities = get_activities(activities_endpoint)

    # Now transform alerts (depends on extracted_alerts being ready)
    transformed_alerts = transform_alerts(extracted_alerts)

    # Save the extracted and transformed data to separate files
    load_into_db(extracted_parks, parks_filename)
    load_into_db(transformed_alerts, alerts_filename)
    load_into_db(extracted_activities, activities_filename)

if __name__ == "__main__":
    scrape_NPS.serve(
        name="NPS Scraper",
        interval = (60*60*24) # once weekly
    )

```

```
!python NPS_scraper.py
```

```

# Import necessary libraries
from prefect.artifacts import create_markdown_artifact, create_link_artifact
from urllib.parse import urljoin
from bs4 import BeautifulSoup
from prefect import flow, task
from pydantic import HttpUrl
from typing import List
import pandas as pd
import datetime
import requests

```

```

import re
# for map
from branca.colormap import LinearColormap
from folium.plugins import MarkerCluster
from ftplib import FTP_TLS
import folium

# Function to load and merge the datasets (parks, alerts, and activities)
@task
def load_and_merge(
    parks_file: str,
    alerts_file: str,
    activities_file: str
) -> pd.DataFrame:
    """
    Loads and merges park, alert, and activity data from the specified JSONL files.

    Args:
        parks_file (str): Path to the parks data file (in JSONL format).
        alerts_file (str): Path to the alerts data file (in JSONL format).
        activities_file (str): Path to the activities data file (in JSONL format).

    Returns:
        pd.DataFrame: A merged DataFrame containing parks, alerts, and activities data.
    """
    # Read in the JSONL files containing parks, alerts, and activities data
    parks_df = pd.read_json(parks_file, lines=True)
    alerts_df = pd.read_json(alerts_file, lines=True)
    activities_df = pd.read_json(activities_file, lines=True)

    # Categorize the designations of parks
    parks_df = categorize_designations(parks_df)

    # Merge parks data with activities data based on park code
    merged_df = pd.merge(parks_df, activities_df, on="parkCode", how="left")

    # Merge the result with the alerts data
    final_df = pd.merge(merged_df, alerts_df, on="parkCode", how="right")

    # Return the final merged DataFrame
    return final_df

```

```

# Function to categorize parks based on their designations
def categorize_designations(parks_df: pd.DataFrame) -> pd.DataFrame:
    """
    Categorizes parks based on their designations into predefined categories.

    Args:
        parks_df (pd.DataFrame): A DataFrame containing park data.

    Returns:
        pd.DataFrame: The input DataFrame with an added 'designation' column for park categorization.
    """
    # Define the categories and corresponding designations
    designation_categories = {
        "National Park or Preserve": [
            'National Park', 'National Parks', 'National Park & Preserve',
            'National Preserve', 'National and State Parks'
        ],
        "Monument or Memorial": [
            'National Monument', 'National Monument and Historic Shrine',
            'National Memorial', 'National Monument & Preserve',
            'Part of Statue of Liberty National Monument'
        ],
        "Historic or Cultural Site": [
            'National Historic Site', 'National Historical Park',
            'National Historical Park and Ecological Preserve',
            'International Historic Site', 'National Historical Park and Preserve',
            'Part of Colonial National Historical Park'
        ],
        "Water-Based or Natural Area": [
            'National Seashore', 'National Lakeshore', 'National River',
            'National Scenic River', 'National Scenic Riverway',
            'National Scenic Riverways', 'National Recreational River',
            'Scenic & Recreational River', 'National River & Recreation Area',
            'National Recreation Area', 'International Park', 'Park'
        ],
        "Trail, Parkway, or Battlefield": [
            'National Historic Trail', 'National Scenic Trail', 'Parkway',
            'Memorial Parkway', 'National Battlefield',
            'National Battlefield Park', 'National Military Park'
        ]
    }
}

```

```

# Create a mapping from designations to categories
designation_map = {
    designation: category
    for category, designations in designation_categories.items()
    for designation in designations
}

# Apply the mapping to categorize the parks
parks_df['designation'] = parks_df['designation'].map(designation_map).fillna('Other')

# Return the updated parks DataFrame
return parks_df

# Function to get the most common alert types from the alerts DataFrame
def get_top_alert_types(df: pd.DataFrame) -> str:
    """
    Extracts the most common alert types from the provided DataFrame.

    Args:
        df (pd.DataFrame): A DataFrame containing alerts data.

    Returns:
        str: A markdown string representing the most common alert types and their counts.
    """
    top_alert_df = df['category'].value_counts().reset_index()
    top_alert_df.columns = ['Category', 'Count']
    top_alert_md = top_alert_df.to_markdown(index=False)
    return top_alert_md

# Function to get the top sites based on a specific metric (e.g., number of alerts or alert s
def get_top_sites(
    df: pd.DataFrame,
    by: str,
    site_type: str,
    top_n: int,
    ascending: bool = False
) -> str:
    """
    Retrieves the top N sites based on a specified metric (e.g., number of alerts or alert s

    Args:
        df (pd.DataFrame): A DataFrame containing site data.

```

```

        by (str): The metric column to sort by (e.g., 'num_alerts', 'score').
        site_type (str): The type of sites to filter by (e.g., 'National Park').
        top_n (int): The number of top sites to retrieve.
        ascending (bool, optional): Whether to sort in ascending order (default is False).

Returns:
    str: A markdown table of the top sites with the specified metric.
"""
# Aggregate the data for the specified site type
agg_df = aggregate_data(df, site_type)

# Keep only relevant columns
keep_cols = ['fullName', 'states', 'url', 'sentiment', 'severity', by]
agg_df = agg_df[keep_cols]

# Sort the DataFrame based on the specified metric (ascending or descending)
sorted_df = agg_df.sort_values(by=by, ascending=ascending).head(top_n)

# Convert the sorted DataFrame to a markdown table
md_table = create_md_table(sorted_df, keep_cols)

return md_table

# Function to aggregate alert data for a specific site type
def aggregate_data(df: pd.DataFrame, site_type: str) -> pd.DataFrame:
    """
    Aggregates alert data for a specific site type and calculates average sentiment, severity,
    total score, and the number of alerts.

    Args:
        df (pd.DataFrame): A DataFrame containing alerts data.
        site_type (str): The site type to filter by (e.g., 'National Park').

    Returns:
        pd.DataFrame: A DataFrame containing aggregated alert data for the specified site type.
    """
    # Filter the DataFrame by the specified site type
    df = df[df['designation'] == site_type]

    # Aggregate the data (average sentiment, severity, total score, and number of alerts)
    agg_df = df.groupby(['fullName', 'states', 'url']).agg({
        'sentiment': 'mean',

```

```

        'severity': 'mean',
        'score': 'sum',
        'fullName': 'size'
    }).rename(columns={'fullName': 'num_alerts'}).reset_index()

    # Normalize the score for readability
    agg_df["score"] = round((agg_df["score"] - min(agg_df["score"]))) / (max(agg_df["score"]))

    return agg_df

# Function to create a markdown table from the sorted DataFrame
def create_md_table(sorted_df: pd.DataFrame, keep_cols: List[str]) -> str:
    """
    Converts the sorted DataFrame into a markdown table.

    Args:
        sorted_df (pd.DataFrame): The sorted DataFrame containing site data.
        keep_cols (List[str]): The list of columns to include in the markdown table.

    Returns:
        str: A markdown-formatted table as a string.
    """
    # Map the column names to user-friendly names
    rename_map = {
        'fullName': 'Site Name',
        'states': 'State(s)',
        'url': 'Site Website',
        'sentiment': 'Avg. Sentiment',
        'severity': 'Avg. Severity',
        'score': 'Alert Score',
        'num_alerts': 'Number of Alerts'
    }
    rename_map = {k:v for k,v in rename_map.items() if k in keep_cols}

    # Rename columns
    sorted_df = sorted_df.rename(columns=rename_map).reset_index(drop=True)

    # Set the index to start from 1 for display purposes
    sorted_df.index = sorted_df.index + 1

    # Convert the DataFrame to a markdown table
    md_table = sorted_df.to_markdown()

```

```

    return md_table

# Function to get top images from a park website
def get_top_images(park_url: HttpUrl) -> str:
    """
    Retrieves the top image from a park's website by scraping the HTML content.

    Args:
        park_url (HttpUrl): The URL of the park's website.

    Returns:
        str: HTML image tag containing the top image for the park.
    """
    # Fetch the webpage content
    page_url = park_url
    response = requests.get(page_url)
    soup = BeautifulSoup(response.text, 'html.parser')

    # Find the div containing the background image
    div = soup.find('div', class_='picturefill-background')
    style = div.get('style')

    # Extract the image URL from the style attribute
    match_url = re.search(r"url\('(.*?)'\)", style)
    relative_img_url = match_url.group(1)

    # Build the full image URL and return HTML for embedding the image
    full_img_url = urljoin('https://www.nps.gov', relative_img_url)
    img_html = f"""\n\n"""

    return img_html

# Function to get popular activities at top sites
def get_top_activities(df: pd.DataFrame, site_type: str, top_n: int = 5) -> str:
    """
    Retrieves the most popular activities at the top sites based on alert scores.

    Args:
        df (pd.DataFrame): A DataFrame containing park and activity data.
        site_type (str): The site type to filter by (e.g., 'National Park').
        top_n (int, optional): The number of top activities to return (default is 5).
    """

```

```

Returns:
    str: A markdown string containing popular activities at top sites.
"""

# Aggregate data to get the top parks based on the score
agg_df = aggregate_data(df, site_type)
top_parks = agg_df.sort_values("score").head(top_n)

# Filter the original DataFrame to get data for top parks
top_park_names = top_parks['fullName'].tolist()
top_df = df[df['fullName'].isin(top_park_names)][['fullName', 'url', 'activity']]

# Prepare markdown content for activities
activity_md = ""
for idx, row in top_df.iterrows():
    activity_md += get_top_images(row['url'])
    activity_md += f"### {row['fullName']}\n"
    for acts in row['activity']:
        activity_md += f"- {acts}\n"

    activity_md += f"\n<br>\n<br>\n"

return activity_md

# task function to generate the full report
@task
def load_into_report(
    df: pd.DataFrame,
    site_type: str,
    top_n: int = 10
) -> str:
    """
    Generates a markdown report summarizing the top alert types, sites with the most alerts,
    sites with the highest alert scores, and popular activities at the top sites.

    Args:
        df (pd.DataFrame): The merged DataFrame containing parks, alerts, and activities data.
        site_type (str): The type of sites to report on (e.g., 'National Park').
        top_n (int, optional): The number of top sites to include in the report (default is 10).

    Returns:
        str: A markdown-formatted report as a string.
    """

```



```

# Calculate the total number of alerts and current date
total_alerts = len(df)
date = datetime.datetime.now().strftime("%d-%b-%Y")

# Get the top alert types, top sites by number of alerts and alert score, and popular ac
top_alert_types= get_top_alert_types(df)
top_alert_sites = get_top_sites(df, 'num_alerts', site_type, top_n)
worst_score_sites = get_top_sites(df, 'score', site_type, top_n)
best_score_sites = get_top_sites(df, 'score', site_type, top_n, ascending=True)
top_activities = get_top_activities(df, site_type)

# Compile all results into a markdown report
report_md = f"""


# NPS Scraping Report

**Report Date:** {date}

**Total Alerts Collected:** {total_alerts}

**Site Type:** {site_type}

<br><br>

## Most Common Alert Types

The most frequently occurring alert types were:

{top_alert_types}

<br><br>

## {site_type} with the Most Alerts

{top_alert_sites}

<br><br>

## {site_type} with Highest Alert Scores

Using a scoring system that adjusts for both alert type and sentiment, we've identified the

```

```

{worst_score_sites}

<br><br>

##  {site_type} with Highest Alert Scores

Conversely, these {top_n} sites are currently facing the fewest and least serious issues:

{best_score_sites}

<br><br>

##  Popular Activities at Top {site_type}

{top_activities}
"""

    create_markdown_artifact(
        markdown=report_md,
        key="nps-report",
        description="Report from todays run of the NPS Scraper"
    )

def map_parks(df: pd.DataFrame, map_filename: str) -> str:
    """
    Generates a folium map visualizing the parks with alert scores, including markers and a

    Args:
        df (pd.DataFrame): The DataFrame containing park and alert data.
        map_filename (str): The filename to save the generated map as an HTML file.

    Returns:
        str: The filename of the saved map.
    """
    # clean df
    df_clean = df.dropna(subset=["latitude", "longitude", "score"])
    # Create custom green-to-red color scale
    min_score = df_clean['score'].min()
    max_score = df_clean['score'].max()
    colormap = LinearColormap(
        colors=['green', 'yellow', 'red'],
        vmin=min_score,

```

```

        vmax=max_score,
        caption='Alert Score (Green = Low, Red = High)'
    )

    # Create folium map centered on the U.S.
    m = folium.Map(location=[39.8283, -98.5795], zoom_start=4)
    marker_cluster = MarkerCluster().add_to(m)

    for _, row in df_clean.iterrows():
        color = colormap(row['score'])
        popup_html = f"""
        <b><a href='{row['url']}' target='_blank'>{row['fullName']}</a></b><br>
        <i>{row['designation']}</i><br>
        <b>State(s):</b> {row['states']}<br>
        <b>Category:</b> {row['category']}<br>
        <b>Sentiment:</b> {row['sentiment']}<br>
        <b>Severity:</b> {row['severity']}<br>
        <b>Score:</b> {row['score']}<br>
        <b>Description:</b><br>{row['description']]
        """
        folium.CircleMarker(
            location=[row['latitude'], row['longitude']],
            radius=5,
            popup=folium.Popup(popup_html, max_width=300),
            tooltip=row['fullName'],
            color=color,
            fill=True,
            fill_color=color,
            fill_opacity=0.7,
        ).add_to(marker_cluster)

    # Add the colormap to the map
    colormap.add_to(m)

    # Save the map to an HTML file
    m.save(map_filename)
    return map_filename

def upload_to_ftp(file_name: str) -> str:
    """
    Uploads the generated map HTML file to an FTP server.

```

```

Args:
    file_name (str): The name of the file to upload.

Returns:
    str: The host URL of the FTP server where the file was uploaded.
"""
# Get FTP credentials (assuming 'userdata' is defined somewhere in your code)
ftp_host = userdata.get("FTP_HOST")
ftp_user = userdata.get("FTP_USER")
ftp_pass = userdata.get("FTP_PASS")

try:
    # Connect to FTP server with TLS (secure connection)
    ftp = FTP_TLS(ftp_host)
    ftp.login(ftp_user, ftp_pass)

    # Secure the connection
    ftp.prot_p()

    # Change to the public_html/5500-project directory
    ftp.cwd('public_html/5500-project')

    # Open file and upload
    with open(file_name, 'rb') as file:
        ftp.storbinary(f"STOR {file_name}", file)

    ftp.quit()
    print(f"Successfully uploaded {file_name} to {ftp_host}")

except Exception as e:
    print(f"An error occurred: {e}")

return ftp_host

# task function to create map
@task
def generate_and_upload_map(df: pd.DataFrame, map_filename: str) -> None:
    """
    Generates the map and uploads it to the FTP server.

    Args:

```

```

    df (pd.DataFrame): The DataFrame containing park and alert data.
    map_filename (str): The filename for the map to be generated and uploaded.
    """
    # Generate the map and save it as an HTML file
    map_filename = map_parks(df, map_filename)

    # Upload the generated map to the FTP server
    host = upload_to_ftp(map_filename)
    # create link for the saved map
    map_link = 'https://' + host + '/5500-project/' + map_filename
    # make it an artifact
    create_link_artifact(
        link=map_link,
        key="nps-map",
        description="Map from todays run of the NPS Scraper"
    )

# flow to generate report and map
@flow(name="NPS Report")
def generate_report(
    site_type: str = "National Park or Preserve",
    parks_filepath: str = "extracted_parks.jsonl",
    alerts_filepath: str = "extracted_alerts.jsonl",
    activities_filepath: str = "extracted_activities.jsonl",
    map_filename: str = "map.html"
) -> None:
    """
    Orchestrates the process of loading data, generating a report, and creating a map.

    Args:
        site_type (str, optional): The type of site to report on (default is 'National Park or Preserve')
        parks_filepath (str, optional): Path to the parks data file (default is 'extracted_parks.jsonl')
        alerts_filepath (str, optional): Path to the alerts data file (default is 'extracted_alerts.jsonl')
        activities_filepath (str, optional): Path to the activities data file (default is 'extracted_activities.jsonl')
        map_filename (str, optional): The filename for the generated map (default is 'map.html')
    """
    # Load and merge the data
    df = load_and_merge(parks_filepath, alerts_filepath, activities_filepath)
    # Generate the report
    load_into_report(df=df, site_type=site_type)
    # generate map
    generate_and_upload_map(df=df, map_filename=map_filename)

```

```
generate_report()
```